NEORV32 Accelerator for Cryptography

Ascon-AEAD128 Hardware Accelerator

Requirements & Design Document

Author	Mohammadi Masoudi Ramin
Project Title	NEORV32 Accelerator for Cryptography
Target Platform	NEORV32 RISC-V SoC on Sipeed Tang Nano 9K
FPGA Device	Gowin GW1NR-9 (≈8876 LUTs, 6480 FFs)
Integration	Custom Functions Subsystem (CFS)
Algorithm	Ascon-AEAD128 per NIST SP 800-232 final (August 2025)
HDL	Verilog (MIT licence)
Document Rev.	0.5
Date	May 20, 2026

This document defines the functional and non-functional requirements, hardware architecture, register map, firmware programming model, verification strategy, and validation plan for a hardware accelerator implementing the NIST Ascon-AEAD128 authenticated encryption algorithm, integrated into a NEORV32 RISC-V processor system via the Custom Functions Subsystem.

Contents

1 Introduction

1.1 Purpose

This document specifies the requirements and design of a hardware accelerator implementing the Ascon-AEAD128 lightweight authenticated encryption algorithm. It is the technical reference for the project *NEORV32 Accelerator for Cryptography*, covering algorithm background, system requirements, hardware architecture, register-level programming model, verification and validation approach, design challenges, and directions for future work.

1.2 Project Goals

The project explores four complementary areas:

1. **Efficient hardware implementation of cryptographic primitives:** design a synthesisable Verilog implementation of the full Ascon-AEAD128 algorithm, structured for readability and FPGA resource efficiency.
2. **CPU-to-accelerator interface:** expose the accelerator to software through a memory-mapped register interface compatible with the NEORV32 Custom Functions Subsystem.
3. **Correctness verification:** validate the RTL against a C golden model and a self-checking testbench suite covering primitives, AEAD core, and MMIO wrapper.
4. **Performance benchmarking:** measure FPGA resource usage and compare hardware versus software throughput on the same processor.

1.3 Design Status

This RASD describes the intended NEORV32 accelerator architecture and the requirements that the implementation SHALL satisfy. The current development flow uses a typed Python golden model, known-answer tests, generated Verilog helpers, and Tang Nano 9K board bring-up infrastructure to reduce ambiguity before final NEORV32 integration. Final synthesis, on-board validation, and benchmark results will be reported in a separate development report.

1.4 Scope

The document covers the complete Ascon-AEAD128 accelerator:

- The 320-bit Ascon permutation ($p[8]$ and $p[12]$).
- The AEAD128 mode: initialisation, associated-data absorption, plaintext/ciphertext streaming, finalisation, tag generation, and tag verification.
- The 32-bit MMIO register wrapper.
- The NEORV32 CFS VHDL integration wrapper.
- The C firmware driver.

Out of scope: ASIC physical implementation, ASIC synthesis, physical layout, advanced side-channel countermeasures, DMA, multi-round-per-cycle datapaths, and cryptographic certification. Architecture exploration beyond the NEORV32 FPGA accelerator is documented separately in the development report.

1.5 Assumptions and Constraints

- **Target board:** Sipeed Tang Nano 9K (Gowin GW1NR-9).
- **Integration point:** NEORV32 Custom Functions Subsystem.
- **HDL:** Verilog, synthesisable subset, no vendor primitives.
- **Development environment:** Nix flake providing Icarus Verilog, Yosys, GTKWave, GNU Make, Python 3.
- **Reference model:** the `ascon-c` C reference implementation is used as the golden model for vector generation and cross-validation. The Python scripts in `tools/` are supplementary utilities for bus-formatted vector printing.
- **Architecture:** iterative permutation, one round per clock cycle, prioritising area and design clarity over throughput.

1.6 Normative Language

The key words **SHALL**, **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are interpreted as described in RFC 2119.

1.7 References

1. NIST SP 800-232, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices*, final publication, August 2025.
2. Dobraunig et al., *Ascon v1.2*, Journal of Cryptology, 2021.
3. NEORV32 Processor Datasheet and User Guide, https://stnolting.github.io/neorv32/
4. NEORV32 Custom Functions Subsystem documentation and CFS driver header.
5. Sipeed Tang Nano 9K Schematic, Gowin GW1NR-9 Datasheet.
6. Ascon-c reference implementation, https://github.com/ascon/ascon-c

1.8 Terminology

Term	Definition
AEAD	Authenticated Encryption with Associated Data
CFS	Custom Functions Subsystem — NEORV32's memory-mapped accelerator interface
IV	Initialisation Vector — 64-bit constant encoding Ascon-AEAD128 parameters
KAT	Known Answer Test
LUT	Look-Up Table, basic logic element of an FPGA
FF	Flip-Flop, single-bit synchronous storage element
$p[n]$	Ascon permutation applied for n rounds
Rate	State bits absorbed or squeezed per block (128 bits = 16 bytes)
Tag	128-bit authentication code appended to ciphertext
RTL	Register Transfer Level hardware description
DUT	Device Under Test

2 Background: Ascon-AEAD128

2.1 Why Ascon?

Ascon is a family of lightweight cryptographic algorithms selected in 2023 as NIST's primary standard for lightweight cryptography (SP 800-232). Its design philosophy is simplicity: the 320-bit state fits in five 64-bit registers, and the entire permutation is built from XOR, AND, NOT, and fixed rotations. This makes it unusually compact in hardware while remaining secure at 128-bit equivalent strength. Although the course assignment lists AES and SHA as representative examples, Ascon-AEAD128 satisfies the same accelerator goal: it is a standard cryptographic primitive implemented in hardware, exposed through a CPU interface, verified against known-answer tests, and benchmarked against software execution.

2.2 State Representation

The Ascon state is a vector of five 64-bit words:

$$S = (S_0 \parallel S_1 \parallel S_2 \parallel S_3 \parallel S_4)$$

In the Verilog implementation this maps directly to a packed 320-bit bus:

State word	Bit slice
S_0	`state[63:0]`
S_1	`state[127:64]`
S_2	`state[191:128]`
S_3	`state[255:192]`
S_4	`state[319:256]`

Consequently, when a Verilog concatenation is used to build the packed bus, the word order is reversed relative to the mathematical list:

$$\texttt{state} = \{S_4, S_3, S_2, S_1, S_0\}.$$

External byte strings are interpreted little-endian within each 64-bit word: byte 0 is the least-significant byte of S_0. For example, the byte sequence `00 01 02 03 04 05 06 07` is represented as the integer word `0x0706050403020100`. This distinction between byte-string display and integer-word display is mandatory for all RTL, firmware, and test vectors.

2.3 Permutation Structure

Each round applies three sequential layers to the full 320-bit state:

1. **Constant addition (p_C):** XOR an 8-bit round constant into the least-significant byte of S_2. The constant for round i of a k-round permutation is $c_i = \texttt{CONST}[16 - k + i]$, so $p[12]$ uses table indices 4–15 and $p[8]$ uses indices 8–15. The full table (index 0–15) is: `0x3c, 0x2d, 0x1e, 0x0f, 0xf0, 0xe1, 0xd2, 0xc3, 0xb4, 0xa5, 0x96, 0x87, 0x78, 0x69, 0x5a, 0x4b`.
2. **Substitution layer (p_S):** A non-linear 5-bit S-box is applied to all 64 vertical bit-slices of the state simultaneously, using the bit-sliced Boolean circuit given in Appendix A of the NIST specification.

3. **Linear diffusion layer (p_L):** Each word is updated by XOR-ing with two rotated copies of itself:

$$S_0 \leftarrow S_0 \oplus \mathrm{rotR}(S_0, 19) \oplus \mathrm{rotR}(S_0, 28), \qquad S_1 \leftarrow S_1 \oplus \mathrm{rotR}(S_1, 61) \oplus \mathrm{rotR}(S_1, 39),$$
$$S_2 \leftarrow S_2 \oplus \mathrm{rotR}(S_2, 1) \oplus \mathrm{rotR}(S_2, 6), \qquad S_3 \leftarrow S_3 \oplus \mathrm{rotR}(S_3, 10) \oplus \mathrm{rotR}(S_3, 17),$$
$$S_4 \leftarrow S_4 \oplus \mathrm{rotR}(S_4, 7) \oplus \mathrm{rotR}(S_4, 41).$$

2.4 AEAD128 Mode of Operation

Phase 1 — Initialisation

$$S \leftarrow p^{12}\big(IV \parallel K \parallel N\big) \oplus \big(0^{192} \parallel K\big)$$

The initialisation vector $IV =$ `0x00001000808c0001` encodes the algorithm parameters (k=128, rate=128, a=12, b=8). After p[12], the 128-bit key K is XORed back into $S_3 \| S_4$. In the RTL layout this corresponds to `state[319:192]`.

Phase 2 — Associated Data

Associated data A is absorbed in 128-bit blocks: for each block, XOR it into $(S_0 \| S_1)$, then apply p[8]. The final partial block is padded by appending a 1-bit at the next bit position followed by zeroes up to the rate boundary. After the last AD block, the AEAD domain-separation bit is applied by toggling logical bit S[319], which corresponds to the most-significant bit of S_4 in the RTL vector.

Phase 3 — Plaintext/Ciphertext

For each 128-bit plaintext block P_i:

$$C_i = P_i \oplus (S_0 \| S_1), \qquad (S_0 \| S_1) \leftarrow C_i, \qquad \text{then apply } p[8].$$

Decryption reverses the XOR to recover P_i from C_i, while the state absorbs C_i identically to encryption. The final block uses the same padding as AD.

Phase 4 — Finalisation

$$S \leftarrow p^{12}\big(S \oplus (0^{128} \parallel K \parallel 0^{64})\big), \qquad T = (S_3 \oplus K_0) \parallel (S_4 \oplus K_1)$$

where $K_0 = K$[63:0] and $K_1 = K$[127:64]. In the RTL state layout this final key injection updates S_2 and S_3, and tag extraction uses S_3 and S_4.

2.5 Bus Byte Convention

All 128-bit external buses use a **little-endian byte** layout: `bus[7:0]` = byte 0. MMIO registers expose the low 32-bit word first (`KEY0` holds key bytes 0–3). The C driver, testbenches, and reference scripts all adhere to this convention; any deviation produces incorrect ciphertext with no other diagnostic.

3 System Requirements

3.1 Functional Requirements

FR-001 | *State representation*

The design **SHALL** represent the Ascon state as five 64-bit words S_0–S_4 packed into a 320-bit vector using the NIST little-endian state convention: S_0 at bits `[63:0]`, S_1 at `[127:64]`, S_2 at `[191:128]`, S_3 at `[255:192]`, and S_4 at `[319:256]`.

FR-002 | *Round constant addition*

The design **SHALL** XOR the 8-bit round constant (selected by index $16 - \mathit{rounds} + i$) into the least-significant byte of S_2. Constants **SHALL** match NIST SP 800-232, Table 5, exactly.

FR-003 | *Substitution layer*

The design **SHALL** implement the Ascon S-box as a bit-sliced Boolean circuit applied to all 64 vertical bit-slices of the state in parallel. The truth table **SHALL** agree with the reference specification for all 32 possible inputs.

FR-004 | *Linear diffusion layer*

The design **SHALL** implement p_L using the XOR-rotation amounts specified for each of the five 64-bit state words.

FR-005 | *One combinational round*

The design **SHALL** compose one complete round as $p_L(p_S(p_C(\text{state})))$ in a purely combinational block, with no internal state registers.

FR-006 | *Iterative permutation engine*

The design **SHALL** implement an iterative engine accepting a `rounds_i` parameter in the range 1–16, advancing one round per clock cycle, and asserting `done_o` for exactly one cycle on completion.

FR-007 | *Permutation variants*

Ascon-AEAD128 **SHALL** use $p[12]$ during initialisation and finalisation, and $p[8]$ during data absorption. The round-constant start index is computed as $16 - k$ for a k-round permutation.

FR-008 | *Authenticated encryption*

The design **SHALL** implement the Ascon-AEAD128 encryption mode per NIST SP 800-232: initialisation, AD absorption with padding and domain separation, plaintext absorption with ciphertext output, and finalisation with 128-bit tag generation.

FR-009 | *Authenticated decryption*

The design **SHALL** implement Ascon-AEAD128 decryption, recovering plaintext from ciphertext and performing a constant-time 128-bit tag comparison. The `tag_ok_o` output **SHALL** be high if and only if all 128 tag bits match.

FR-010 | *Input length support*

The design **SHALL** correctly handle: zero-length AD, zero-length message, inputs that are exact multiples of 16 bytes, and partial final blocks of 1–15 bytes for both AD and message.

FR-011 | *Padding and domain separation*

Padding (1-bit append at the rate boundary) and AD/message domain separation **SHALL** be applied by the hardware internally, without any software pre-processing.

FR-012 | *Tag verification*

The tag comparison in decryption **SHALL** be implemented as a constant-time bitwise AND-reduction over all 128 bits. It **MUST NOT** short-circuit on any earlier bit match or mismatch.

FR-013 | *CPU register interface*

The design **SHALL** expose a 32-bit memory-mapped interface compatible with the NEORV32 CFS bus: one-cycle pipelined acknowledge, full-word-only writes enforced at the hardware level, and 8-bit address decoding within the 256-byte CFS window.

FR-014 | *NEORV32 CFS integration*

The design **SHALL** include a VHDL wrapper (`neorv32_cfs.vhd`) presenting the standard NEORV32 CFS entity ports (`bus_req_i`, `bus_rsp_o`, `irq_o`) and instantiating the Verilog MMIO core as a component.

FR-015 | *Authenticated plaintext release*

During decryption, plaintext blocks **SHALL NOT** be released outside the accelerator until the final authentication tag comparison succeeds. The design **SHALL** buffer recovered plaintext internally during decryption. If tag verification fails, the buffered plaintext **SHALL** be cleared or invalidated and `STATUS.TAG_OK` **SHALL** remain de-asserted.

3.2 Non-Functional Requirements

NFR-001 | *Synthesisability*

The RTL **SHALL** elaborate under Yosys without errors, produce a valid JSON netlist, and contain no combinational loops and no vendor-specific primitives.

NFR-002 | *Iterative architecture*

The initial implementation **SHALL** apply one round per clock cycle. This choice favours area and design transparency; higher-performance variants are deferred to future work.

NFR-003 | *Reproducible toolchain*

The repository **SHALL** provide a `flake.nix` and `shell.nix` that supply a fully reproducible environment: Icarus Verilog, Yosys, Graphviz, GTKWave, Python 3, and GNU Make.

NFR-004 | *Testbench suite*

Self-checking testbenches **SHALL** be provided at the primitive, AEAD core, and MMIO wrapper levels. Each **SHALL** exit with a non-zero status code on failure and print an unambiguous `PASS` or `FAIL` line.

NFR-005 | *Firmware driver correctness*

The C driver **SHALL** implement a polling-based communication loop covering both encryption and decryption through a shared internal function. Interrupt-driven operation is optional.

NFR-006 | *Core decoupling*

`ascon_aead128_core` **SHALL** contain no NEORV32-specific signals. All processor-side integration **SHALL** be confined to `ascon_aead128_mmio` and the VHDL wrapper, so the core can be reused with any 128-bit streaming interface.

NFR-007 | *Timing determinism*

Execution latency for a fixed input length **SHALL NOT** depend on the secret key value, plaintext, ciphertext, or tag content. The control path **SHALL** be identical for all data patterns.

4 Hardware Architecture

4.1 Layered Design

The design separates concerns into four layers:

1. **Permutation primitive layer:** round constants, substitution, diffusion, a single combinational round, and the iterative permutation engine.
2. **AEAD mode layer:** the streaming controller implementing all four algorithm phases.
3. **MMIO layer:** the 32-bit register file and CFS bus interface.
4. **Integration layer:** the VHDL CFS wrapper and C firmware driver.

The call chain from firmware down to the permutation primitive is:

```
NEORV32 C firmware
  → CFS address space (NEORV32 bus)
    → neorv32_cfs.vhd
      → ascon_aead128_mmio.v
        → ascon_aead128_core.v
          → ascon_permutation_iter.v
            → ascon_round.v
```

4.2 Top-Level Block Diagram

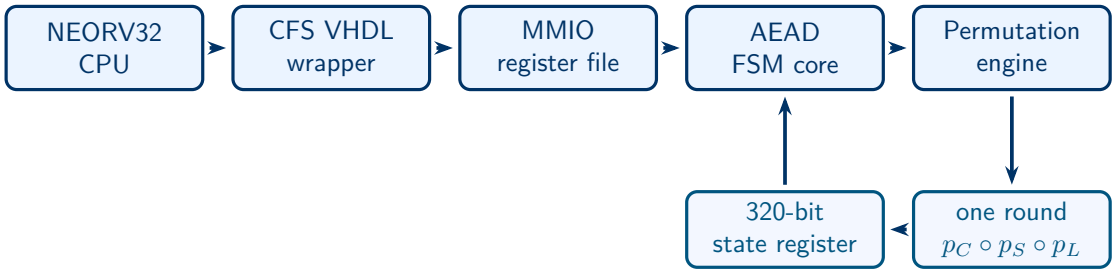

Figure 1: System architecture. The five design layers appear left to right: CPU, CFS VHDL wrapper, MMIO register file, AEAD FSM core, and permutation engine. Below, the 320-bit state register and combinational round function form the per-cycle permutation feedback loop. On completion, the AEAD core asserts an interrupt back to the CPU via `irq_o`.

4.3 Module Hierarchy

Module	File	Responsibility
`ascon_round_constant`	`ascon_round_constant.v`	4-bit index to 8-bit constant ROM (16 entries per NIST SP 800-232 Table 5).
`ascon_constant_add`	`ascon_constant_add.v`	XOR constant into LSB of S_2 (p_C layer).
`ascon_sbox5`	`ascon_sbox5.v`	Standalone 5-bit S-box truth-table LUT for formal equivalence checking.
`ascon_substitution`	`ascon_substitution.v`	Bit-sliced Boolean p_S applied to all 64 slices in parallel.
`ascon_diffusion`	`ascon_diffusion.v`	XOR-rotation diffusion layer p_L for S_0–S_4.
`ascon_round`	`ascon_round.v`	One purely combinational round: $p_L(p_S(p_C(\cdot)))$.
`ascon_state_reg`	`ascon_state_reg.v`	320-bit register with explicit load/update control and word outputs.
`ascon_permutation_iter`	`ascon_permutation_iter.v`	Iterative engine: one round per clock, round counter, done signal.
`ascon_aead128_core`	`ascon_aead128_core.v`	AEAD streaming FSM: all four algorithm phases, padding, tag comparison.
`ascon_aead128_mmio`	`ascon_aead128_mmio.v`	32-bit register wrapper, bus decode, IRQ, CFS protocol.
`neorv32_cfs`	`neorv32_cfs.vhd`	VHDL drop-in replacement for the NEORV32 CFS template.

4.4 Permutation Engine

The iterative engine (`ascon_permutation_iter`) processes one combinational round per clock cycle:

- On `start_i`: latch `state_i` and `rounds_i`; assert `busy_o`; reset the round counter to zero.
- Each cycle: pass `state_o` through one `ascon_round` instance and write the result back into `state_o`; increment the round counter.
- When the counter reaches the target: de-assert `busy_o`, pulse `done_o` for one cycle.

The first constant index is $16 - \texttt{rounds\_i}$, ensuring $p[12]$ selects table entries 4–15 and $p[8]$ selects entries 8–15, exactly as the specification requires.

Latency: 12 cycles for $p[12]$; 8 cycles for $p[8]$.

4.5 AEAD Core FSM

The AEAD controller (`ascon_aead128_core`) sequences through 16 states arranged in four algorithmic phases, each corresponding to a column in Figure 2.

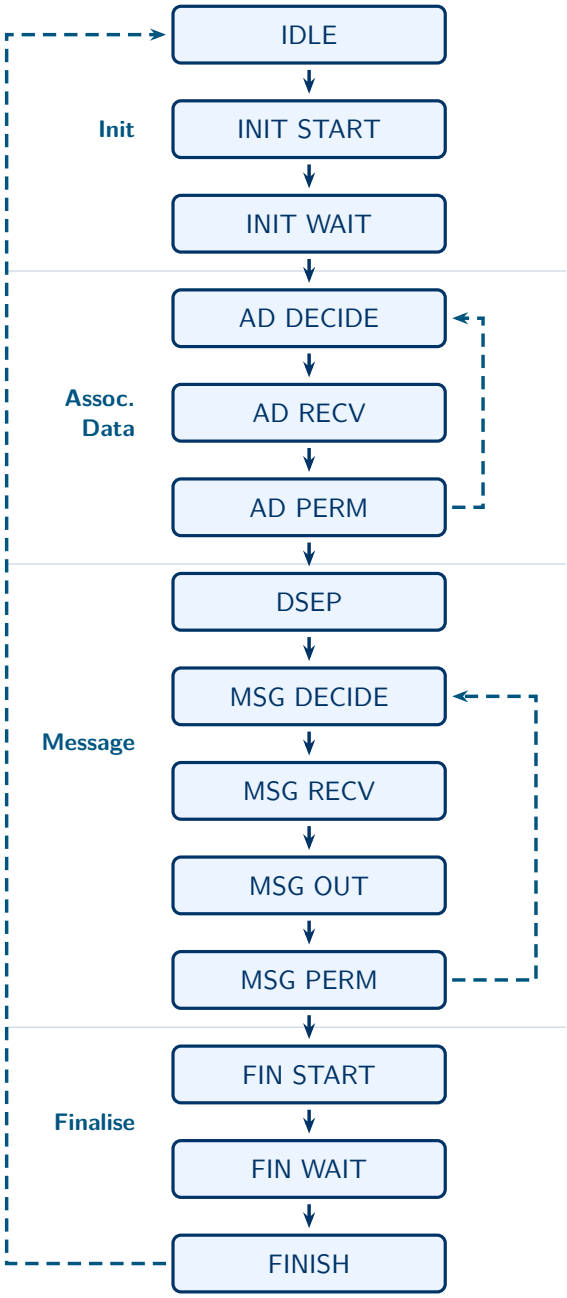

Figure 2: AEAD core FSM. States appear top to bottom in execution order. Phase boundaries are marked by light horizontal rules. Dashed arcs on the right: per-block iteration loops back to `AD DECIDE` and `MSG DECIDE`. Dashed arc on the left: return to `IDLE` on completion. Every state is described in full in the table below.

State	Duration	Action
ST_IDLE	—	Wait for start_i. Latch key, nonce, mode, lengths, and expected tag.
ST_INIT_START	1 cycle	Load state = $(IV \| K \| N)$; pulse permutation with rounds = 12.
ST_INIT_WAIT	12 cycles	Wait for $p[12]$ done; XOR key into S_3, S_4.
ST_AD_DECIDE	1 cycle	Determine whether an AD block is pending or AD processing is complete.
ST_AD_RECV	varies	Wait for a valid AD block; XOR into $(S_0 \| S_1)$ with padding logic.
ST_AD_PERM_START	1 cycle	Pulse permutation with rounds = 8.
ST_AD_PERM_WAIT	8 cycles	Wait for $p[8]$ done; update state register.
ST_DSEP	1 cycle	XOR domain-separation word into S_4.
ST_MSG_DECIDE	1 cycle	Determine whether a message block or padding-only step is next.
ST_MSG_RECV	varies	Accept PT/CT block; compute CT/PT output; update state with padding logic.
ST_MSG_OUT	1+ cycles	Encryption: hold data_valid_o until the consumer acknowledges. Decryption: write recovered plaintext into the internal release buffer.
ST_MSG_PERM_START	1 cycle	Pulse $p[8]$ for a non-final message block.
ST_MSG_PERM_WAIT	8 cycles	Wait for $p[8]$ done.
ST_FINAL_PERM_START	1 cycle	XOR key into state; pulse $p[12]$.
ST_FINAL_PERM_WAIT	12 cycles	Wait for $p[12]$ done; extract tag; compare tag.
ST_FINISH	1 cycle	Assert done_o; return to ST_IDLE.

4.6 Encryption vs. Decryption Data Flow

Step	Encryption	Decryption
Input (MSG phase)	Plaintext P from CPU	Ciphertext C from CPU
Output (MSG phase)	$C_i = P_i \oplus (S_0 \| S_1)$	$P_i = C_i \oplus (S_0 \| S_1)$ is recovered internally and released only after tag verification
State absorption	$(S_0 \| S_1) \leftarrow C_i$	$(S_0 \| S_1) \leftarrow C_i$
Tag production	Generated T; tag_ok_o = 1 always	Computed T compared to TAG_IN
tag_ok_o	Always asserted	High iff all 128 bits match

Important: During decryption, recovered plaintext remains inside the accelerator until the finalisation permutation and tag comparison complete. The hardware **MUST NOT** assert plaintext output validity unless STATUS.DONE and STATUS.TAG_OK are both true. On tag failure, the internal plaintext buffer is invalidated or cleared before returning to idle.

5 Register Map and Programming Model

5.1 Base Address and Access Rules

The accelerator is mapped into the NEORV32 CFS address window. The base address is defined in the NEORV32 software package as `NEORV32_CFS_BASE` (current releases: `0xFFEB0000`). The C driver header uses a `#ifndef` guard so the framework value always takes precedence.

All accesses **MUST** be 32-bit aligned full-word reads or writes. A sub-word write (`be_i` $\neq$ `4'b1111`) is rejected: `err_o` is asserted and `STATUS.ERROR` is latched.

5.2 Complete Register Map

Offset	Name	R/W	Description
0x00	CTRL	R/W	**Write:** bit 0 = START (self-clearing; ignored and sets ERROR if busy), bit 1 = DECRYPT (0 = encrypt, 1 = decrypt), bit 2 = CLEAR (clear sticky flags: done, tag_valid, error). **Read:** bit 1 reflects the latched decrypt mode.
0x04	STATUS	RO	bit 0 = BUSY, bit 1 = DONE, bit 2 = INPUT_READY, bit 3 = INPUT_PENDING, bit 4 = OUTPUT_VALID, bit 5 = TAG_VALID, bit 6 = TAG_OK, bit 7 = ERROR, bits 9:8 = PHASE (00 none, 01 AD, 10 MSG), bit 16 = IRQ.
0x08	AD_LEN	R/W	Associated-data byte count. Write before START.
0x0C	MSG_LEN	R/W	Plaintext/ciphertext byte count. Write before START.
0x10	KEY0	R/W	Key bits [31:0] (bytes 0–3).
0x14	KEY1	R/W	Key bits [63:32] (bytes 4–7).
0x18	KEY2	R/W	Key bits [95:64] (bytes 8–11).
0x1C	KEY3	R/W	Key bits [127:96] (bytes 12–15).
0x20	NONCE0	R/W	Nonce bits [31:0].
0x24	NONCE1	R/W	Nonce bits [63:32].
0x28	NONCE2	R/W	Nonce bits [95:64].
0x2C	NONCE3	R/W	Nonce bits [127:96].
0x30	TAG_IN0	R/W	Expected tag bits [31:0] (decryption only).
0x34	TAG_IN1	R/W	Expected tag bits [63:32].
0x38	TAG_IN2	R/W	Expected tag bits [95:64].
0x3C	TAG_IN3	R/W	Expected tag bits [127:96].
0x40	DIN0	R/W	Input block staging, bits [31:0].
0x44	DIN1	R/W	Input block staging, bits [63:32].
0x48	DIN2	R/W	Input block staging, bits [95:64].
0x4C	DIN3	R/W	Input block staging, bits [127:96].
0x50	DOUT0	RO	Output block, bits [31:0]. Valid when OUTPUT_VALID = 1; for decryption this occurs only after tag verification.

Offset	Name	R/W	Description
0x54	DOUT1	RO	Output block, bits [63:32].
0x58	DOUT2	RO	Output block, bits [95:64].
0x5C	DOUT3	RO	Output block, bits [127:96].
0x60	TAG_OUT0	RO	Generated tag, bits [31:0]. Valid when DONE = 1.
0x64	TAG_OUT1	RO	Generated tag, bits [63:32].
0x68	TAG_OUT2	RO	Generated tag, bits [95:64].
0x6C	TAG_OUT3	RO	Generated tag, bits [127:96].
0x70	DOUT_BYTES	RO	Valid byte count in DOUT (0–16).
0x74	CMD	WO	bit 0 = PUSH (submit DIN block to core), bit 1 = POP (acknowledge DOUT block), bit 2 = CLEAR (clear sticky flags).
0xFC	ID	RO	Device identifier 0x4E435341 (ASCII "ASCN", little-endian word).

5.3 IRQ Sources

irq_o is the OR of three sticky conditions: DONE = 1, OUTPUT_VALID = 1, or ERROR = 1.

5.4 Firmware Programming Sequence

Encryption

1. Write key to KEY0–KEY3 (byte 0 in KEY0[7:0]).
2. Write nonce to NONCE0–NONCE3.
3. Write AD_LEN and MSG_LEN.
4. Write CTRL = 0x00000001 (START, encrypt mode).
5. Poll STATUS in a loop:
   - If INPUT_READY: check PHASE (01 = AD, 10 = MSG). Write next 16-byte block (zero-padded if partial) to DIN0–DIN3, then write CMD = 0x01 (PUSH).
   - If OUTPUT_VALID: read DOUT0–DOUT3 and DOUT_BYTES, then write CMD = 0x02 (POP).
6. When DONE = 1: read TAG_OUT0–TAG_OUT3.

Decryption

Same sequence, except: write CTRL = 0x00000003 (START + DECRYPT) and write the expected tag to TAG_IN0–TAG_IN3 before START. During decryption the hardware buffers recovered plaintext internally. Firmware reads plaintext output only after DONE and STATUS.TAG_OK = 1.

5.5 Firmware Driver API

The driver is implemented in sw/neorv32_ascon/ascon_hw.c with its public interface declared in ascon_hw.h. The implementation consists of three layers: a thin register accessor using little-endian load32_le/store32_le helpers; a block transfer layer managing the PUSH/POP handshake; and a shared internal function run_operation from which both public entry points are derived.

Public API:

Listing 1: Firmware driver public interface (`ascon_hw.h`)

```c
/*
 * Initialise and identify the accelerator.
 * Returns the 32-bit device ID (expected: 0x4E435341).
 */
uint32_t ascon_hw_read_id(void);

/* Clear sticky status flags. Call once before each operation. */
void ascon_hw_clear(void);

/*
 * Authenticated encryption.
 *
 * key         [in]  16-byte secret key.
 * nonce       [in]  16-byte nonce. MUST be unique for every (key, message)
     pair.
 * ad / ad_len[in]  Associated data (may be NULL when ad_len == 0).
 * pt / pt_len[in]  Plaintext input.
 * ct          [out] Ciphertext output (caller allocates pt_len bytes).
 * tag         [out] 16-byte authentication tag.
 *
 * Returns:  0   success
 *          -1   hardware error (STATUS.ERROR set)
 *          -3   guard timeout
 */
int ascon_hw_encrypt(
    const uint8_t key[16], const uint8_t nonce[16],
    const uint8_t *ad,     uint32_t ad_len,
    const uint8_t *pt,     uint32_t pt_len,
    uint8_t *ct,           uint8_t tag[16]);

/*
 * Authenticated decryption.
 *
 * Plaintext is released to the caller only after the hardware has
 * verified the authentication tag. If verification fails, the hardware
 * invalidates its internal plaintext buffer and this function returns -2.
 *
 * Returns:  0   success, tag verified
 *          -1   hardware error
 *          -2   tag mismatch -- ciphertext is invalid or was tampered with
 *          -3   guard timeout
 */
int ascon_hw_decrypt(
    const uint8_t key[16], const uint8_t nonce[16],
    const uint8_t *ad,     uint32_t ad_len,
    const uint8_t *ct,     uint32_t ct_len,
    const uint8_t tag[16],
    uint8_t *pt);
```

6 NEORV32 CFS Integration

6.1 CFS Bus Protocol

Signal	Direction	Description
`bus_req_i.stb`	in	Strobe: valid access, asserted for exactly one cycle.
`bus_req_i.rw`	in	Direction: 1 = write, 0 = read.
`bus_req_i.ben`	in	Byte enables [3:0]. Must be `4'b1111` for writes.
`bus_req_i.addr`	in	Address [31:0]; low 8 bits decoded by the accelerator.
`bus_req_i.data`	in	Write data [31:0].
`bus_rsp_o.ack`	out	Acknowledge asserted in the cycle after `stb`.
`bus_rsp_o.err`	out	Bus error on sub-word write.
`bus_rsp_o.data`	out	Read data [31:0].
`irq_o`	out	Interrupt: DONE ∨ OUTPUT_VALID ∨ ERROR.

6.2 VHDL Wrapper

`integrations/neorv32/neorv32_cfs.vhd` replaces the default NEORV32 CFS template with no changes to the entity name or port signature. It declares `ascon_aead128_mmio` as a VHDL component and maps CFS bus fields directly to MMIO ports. The unused `cfs_out_o` conduit is driven to all-zero.

6.3 Integration Checklist

1. Build and boot an unmodified NEORV32 Tang Nano baseline.
2. Enable `IO_CFS_EN => true` in the NEORV32 top configuration.
3. Replace `rtl/core/neorv32_cfs.vhd` with the Ascon wrapper.
4. Add all `rtl/*.v` files to the FPGA synthesis project.
5. Confirm Gowin EDA accepts mixed VHDL + Verilog elaboration.
6. Add `ascon_hw.c` and `ascon_hw.h` to the firmware project.
7. Flash and run `example_main.c`; expected UART output: `ASCON HW test - PASS`.

7 Verification

Verification answers the question: *does the implementation match its specification?* The strategy is layered, moving from individual primitives up to the complete register interface.

7.1 Verification Layers

Layer	Tool	What is checked
Unit — round constant	Icarus Verilog (`tb_round_constant`)	All 16 constants compared against NIST SP 800-232 Table 5.
Unit — S-box	Icarus Verilog (`tb_sbox5`)	Exhaustive truth table (all 32 inputs).
Unit — layers	Icarus Verilog (`tb_layers`)	p_C, p_S, p_L individually and one combined round against C-model reference values.
Integration — permutation	Icarus Verilog (`tb_permutation_ite`	$p[12]$ and $p[8]$ applied to reference states; output compared against C golden model.
Integration — AEAD core	Icarus Verilog (`tb_aead128_core`)	Eight test cases covering all message/AD length categories.
Integration — MMIO	Icarus Verilog (`tb_mmio`)	Full register-bus encrypt sequence; CMD.PUSH/POP handshake; IRQ.
Formal smoke check	SymbiYosys (`formal/sbox5_equiv`	Proves `ascon_substitution` equals `ascon_sbox5` for all 2^5 inputs using an `anyconst` 5-bit variable.
Synthesis check	Yosys	RTL elaboration, optimisation, and JSON netlist generation without error.

7.2 AEAD Core Test Cases

Test	AD (bytes)	Message (bytes)	What it checks
`enc_empty`	0	0	Tag-only output; no data path exercised.
`enc_16_16`	16	16	One full AD block; one full PT block; CT and tag.
`dec_16_16`	16	16	Decrypt CT from above; recover PT; tag_ok = 1.
`enc_partial`	3	5	Partial AD and PT; padding correctness.
`dec_partial`	3	5	Decrypt partial CT; recover PT; tag_ok = 1.
`enc_multiblock`	19	21	Two AD blocks; two PT blocks; CT and tag.
`dec_multiblock`	19	21	Decrypt two CT blocks; recover PT; tag_ok = 1.
`dec_wrong_tag`	3	5	One flipped tag bit; tag_ok = 0 confirmed.

7.3 Known Answer Test Procedure

For each KAT vector $(K, N, A, P, C_{\text{exp}}, T_{\text{exp}})$ derived from the `ascon-c` reference implementation:

1. Drive inputs to the DUT and assert `start_i`.
2. Capture each output block as `data_valid_o` is raised.

3. After `done_o`: compare C_{DUT} to C_{exp} and T_{DUT} to T_{exp} bit-for-bit; any mismatch is a hard failure.
4. Re-run as decryption with the computed C; verify $P_{\text{out}} = P$ and `tag_ok_o` $= 1$.
5. Flip one bit of T_{exp}; verify `tag_ok_o` $= 0$.

7.4 Build and Test Commands

Command	Action
`make test`	Run all Icarus Verilog testbenches; every line must read `PASS`.
`make lint`	Verilator `--lint-only` static check.
`make synth`	Yosys synthesis of the AEAD core; outputs JSON netlist.
`make synth-mmio`	Yosys synthesis of the MMIO wrapper; outputs JSON netlist.
`make schematic`	Yosys + Graphviz SVG of the AEAD core.
`make schematic-mmio`	SVG of the MMIO wrapper.
`make vectors`	Print bus-formatted reference vectors from the Python tool.
`make waves`	Open `build/tb_aead128_core.vcd` in GTKWave.
`make clean`	Remove all generated build artefacts.

8 Validation

Validation answers a different question: *does the implemented system do what is actually needed in the target environment?* Passing simulation testbenches is necessary but not sufficient. Hardware behaviour under real bus timing, synthesis tool idioms, and actual firmware execution can all reveal issues that simulation misses.

8.1 On-Board Functional Validation

The target scenario is a NEORV32 SoC running on the Tang Nano 9K, with firmware loading over UART. The minimum validation sequence is:

1. **ID register check.** Read `0xFC`; confirm the value is `0x4E435341`. This confirms the CFS address map is correct and the mixed VHDL/Verilog project elaborated cleanly.
2. **Single known-answer encrypt.** Run one KAT vector through the C driver (`ascon_hw_encrypt`); compare the tag over UART against the C golden model. A match confirms the full data path from firmware to register to permutation to tag output is correct.
3. **Encrypt-then-decrypt round trip.** Encrypt a short message and immediately decrypt the result; confirm plaintext recovery and `STATUS.TAG_OK` $= 1$.
4. **Wrong-tag rejection.** Flip one byte of the tag before calling `ascon_hw_decrypt`; confirm it returns -2 (tag mismatch) and the plaintext buffer is discarded.
5. **Error handling.** Attempt a start while busy and verify the `STATUS.ERROR` bit; verify the accelerator recovers after a `CTRL.CLEAR`.

8.2 Cross-Validation Against the C Reference

All KAT vectors in the testbench are derived from the `ascon-c` reference implementation compiled with identical parameters. The Python scripts in `tools/` reproduce the same computation in a dependency-free form for bus-format vector printing. Discrepancies between the RTL output, the C model, and the Python tool indicate a byte-ordering or padding error at a specific boundary.

The validation target is full agreement across all provided vectors before board bring-up is considered complete.

8.3 Resource and Timing Validation

- Run Gowin vendor synthesis on the complete mixed-language project (NEORV32 + Ascon CFS) and record LUT, FF, and BRAM utilisation.
- Confirm timing closure at the target 27 MHz system clock with positive setup slack.
- Measure the accelerator contribution to area separately from the NEORV32 baseline by differencing two synthesis reports.

8.4 Performance Validation

Using the NEORV32 cycle counter (`mcycle` CSR):

1. Measure cycles for a software-only Ascon-AEAD128 encrypt of 64 bytes, 256 bytes, and 1024 bytes using a C reference implementation running on the same CPU.
2. Measure cycles for the same payloads via `ascon_hw_encrypt`, including all register read-/write overhead.
3. Compute the speedup ratio. The target is $\geq 2\times$ for payloads of 64 bytes and above (NFR-002 / PR-01).

9 Design Challenges

9.1 Byte Ordering

The gap between the NIST byte-string representation and a Verilog bus is the most common source of subtle errors. A correct implementation with a single flipped endianness assumption produces wrong ciphertext with no other visible symptom. The chosen solution — a documented little-endian bus convention applied uniformly across the RTL, testbenches, and C driver — eliminates ambiguity at every boundary. Violations are caught by the cross-validation against `ascon-c`.

9.2 Authenticated Decryption Output Buffering

An AEAD mode with a streaming hardware interface faces an inherent tension: the authentication tag cannot be computed until the entire ciphertext has been absorbed, but recovered plaintext is available internally earlier. This design chooses the safer policy: plaintext is buffered inside the accelerator and is released only after tag verification succeeds. If verification fails, the buffer is cleared or invalidated. This increases memory requirements, but it prevents unauthenticated plaintext from escaping through firmware mistakes or protocol misuse.

9.3 Mixed-Language Integration

The NEORV32 ecosystem is entirely VHDL. Wrapping a Verilog core in a VHDL component declaration and connecting them through a commercial toolchain requires careful type-matching: VHDL's `std_ulogic` versus Verilog's `wire`, bus-width consistency, and correct use of the toolchain's mixed-language elaboration mode. The component declaration in `neorv32_cfs.vhd` must be kept exactly in sync with the Verilog module port list to avoid silent mismatch errors during elaboration.

9.4 Area Budget

The GW1NR-9 provides approximately 8876 LUTs total. A minimal NEORV32 configuration already occupies 3500–4500 LUTs, leaving limited headroom. The one-round-per-cycle iterative architecture was chosen specifically to minimise combinational logic: the S-box is a bit-sliced Boolean function (not a ROM), and only one copy of the round datapath is instantiated. The achievable area depends on how aggressively Yosys and Gowin optimise the large state multiplexer trees in the AEAD FSM.

9.5 Constant-Time Tag Comparison

A naive byte-by-byte tag comparison introduces a timing side channel that leaks information about how many bytes of a forged tag are correct. The implementation uses a single-cycle bitwise AND-reduction over all 128 tag bits, producing a result that does not depend on the values being compared. This is a straightforward measure; it does not protect against power or electromagnetic analysis of the permutation itself.

9.6 CFS Address Portability

The CFS base address (`NEORV32_CFS_BASE`) has changed across NEORV32 releases and differs between processor configurations. Hard-coding a specific address in the firmware driver would break the design silently on any other NEORV32 version. The `#ifndef` guard in `ascon_hw.h` ensures the framework-provided definition always takes precedence, but it requires the integrator to include the correct NEORV32 headers before the driver header.

10 Future Directions

This document defines the target NEORV32/FPGA accelerator architecture. The following directions are intentionally kept outside the RASD scope and are better documented in the separate development report after implementation and benchmark results are available.

10.1 Higher-Throughput Permutation Variants

The iterative architecture is limited to one permutation round per clock cycle. Two straightforward enhancements are possible:

- **Two-rounds-per-cycle:** unroll two round instances in series within the same combinational path. This halves latency to 6 cycles for $p[12]$ and 4 for $p[8]$, at roughly double the combinational logic cost. Timing closure at 27 MHz needs to be verified.

- **Fully unrolled:** instantiate all 12 (or 8) round stages as a deep combinational pipeline, completing one permutation per cycle. This is unlikely to meet timing on the GW1NR-9 at 27 MHz but may be viable at a lower clock or on a larger device.

10.2 Ascon-Hash256 and Ascon-XOF128

The same permutation engine can support the hash and XOF variants of the Ascon family with only a different mode-layer controller. No changes to the permutation primitives would be required.

10.3 Interrupt-Driven Firmware Driver

The current polling loop keeps the CPU busy while the hardware computes. An interrupt-driven alternative would allow the CPU to perform other work during the 12- or 8-cycle permutation bursts, yielding higher effective system throughput for multi-task workloads.

10.4 DMA-Friendly Interface

For large message payloads, block-by-block register polling has significant CPU overhead. A future extension could add a simple DMA descriptor interface where the CPU writes source/destination addresses and a length, and the accelerator autonomously fetches input and writes output over the processor bus. This would require a more sophisticated MMIO controller and arbitration.

10.5 Side-Channel Countermeasures

The S-box is the only non-linear component and therefore the primary target for power and electromagnetic side-channel attacks. Domain-oriented masking (DOM) is the standard countermeasure for bit-sliced Boolean circuits and could be applied to `ascon_substitution` without touching any other module. This would increase area and latency by a factor proportional to the masking order.

10.6 Formal Verification of the AEAD Core

The current formal check (`sbox5_equiv.sby`) proves only the S-box equivalence. Extending formal verification to the AEAD FSM would allow properties such as liveness (every started operation eventually reaches `ST_FINISH`), mutual exclusion of conflicting states, and the constant-time tag comparison invariant to be proved exhaustively over all possible input sequences.

10.7 Architecture Exploration Report

Broader architecture exploration, including ASIC-oriented datapaths, multiple permutation styles, multi-context FPGA engines, side-channel countermeasures, and implementation-generator decisions, is outside this RASD. Those decisions will be described in a separate development report together with synthesis results, board bring-up observations, and performance measurements.

11 Remaining Milestones

1. Import the official `ascon-c` KAT vectors into the testbench and confirm full agreement.
2. Integrate the CFS wrapper into a NEORV32 Gowin project; confirm the mixed VHDL/Verilog build elaborates cleanly.
3. Build the Tang Nano 9K bitstream; confirm timing closure and record resource utilisation.
4. Bring up the firmware; confirm encrypt KAT over UART.
5. Run the decrypt and wrong-tag tests on hardware.
6. Benchmark hardware versus software throughput; verify the $\geq 2\times$ speedup target.

Revision History

Rev	Date	Author	Summary
0.1	2025	Mohammadi Masoudi Ramin	Initial draft — permutation scope.
0.2	2025	Mohammadi Masoudi Ramin	Extended to full AEAD core.
0.3	2025	Mohammadi Masoudi Ramin	MMIO register map and FSM.
0.5	May 20, 2026	Mohammadi Masoudi Ramin	Revised scope for NEORV32/FPGA submission; updated NIST final reference; corrected little-endian state mapping and finalisation equation; replaced provisional plaintext release with authenticated plaintext buffering; moved ASIC exploration to future development-report scope.